

Universidad del Valle de Guatemala

Facultad de Ingeniería

Task 1

Entrega parcial – Laboratorio 7

CC3104 – Reinforcement Learning

Integrantes

Andy Fuentes — 22944

Diederich Solís — 22952

Diego Linares — 221256

Fecha: jueves 24 de septiembre de 2026

1. Task 1.1: análisis de DQN estándar

1.1(a) Estado continuo de 256 dimensiones

Sí es razonable usar DQN desde el punto de vista arquitectónico: una red puede aproximar $Q(s, a; \mathbf{w})$ aunque $s \in \mathbb{R}^{256}$ no pueda enumerarse en una tabla. Como las 256 entradas son métricas numéricas y no píxeles, usaríamos una red totalmente conectada (MLP), por ejemplo, dos capas ocultas con ReLU y una capa final de cuatro salidas. Cada salida estima el valor de una acción. Es importante normalizar las variables de entrada, porque los conteos de tráfico y otras métricas pueden tener escalas muy distintas.

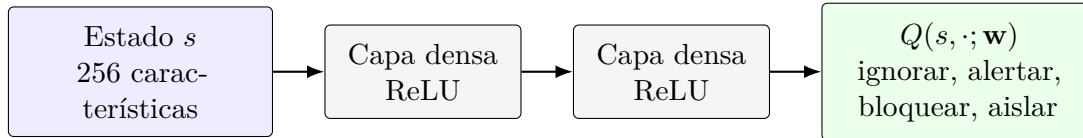


Figura 1: Esquema propuesto para aproximar los cuatro valores Q a partir del vector de estado.

La Figura 1 muestra por qué no hace falta una red convolucional: no se ha definido una estructura espacial como la de una imagen. La conveniencia del método todavía depende de que las características resuman información temporal suficiente y de que exista una forma segura de obtener transiciones y recompensas para aprender.

1.1(b) Cuatro acciones discretas

Las acciones $\mathcal{A} = \{\text{ignorar, alertar, bloquear, aislar}\}$ son finitas y discretas. Por eso DQN y sus variantes pueden producir directamente un valor por acción y elegir

$$a^*(s) = \arg \max_{a \in \mathcal{A}} Q(s, a; \mathbf{w}). \quad (1)$$

No se necesita un algoritmo para acciones continuas. Esta compatibilidad del espacio de acciones no demuestra que DQN sea seguro o eficaz en producción: aislar un segmento tiene un costo distinto al de emitir una alerta, y ambos deben reflejarse en la recompensa y la evaluación.

1.1(c) Eventos críticos raros y Experience Replay

Sea $p < 0.001$ la proporción de pasos con intrusión real. Bajo el supuesto simplificador de eventos independientes y una distribución estable, si el buffer contiene N transiciones, el número K de transiciones críticas satisface $K \sim \text{Binomial}(N, p)$ y $\mathbb{E}[K] = Np$. En tráfico real puede haber ráfagas y cambios de distribución; por ello, esta expresión es una referencia, no una garantía sobre el contenido del buffer. Al ser de capacidad fija, el buffer también expulsa transiciones antiguas: puede llegar a contener $K = 0$ eventos críticos.

Condicionado a un buffer con K eventos críticos, un minibatch uniforme de tamaño B sin reemplazo contiene al menos uno con probabilidad

$$\Pr(\text{al menos un crítico} \mid K) = 1 - \frac{\binom{N-K}{B}}{\binom{N}{B}}, \quad \mathbb{E}[\text{críticos por lote} \mid K] = B \frac{K}{N}. \quad (2)$$

Para ilustrar, si un buffer representativo tiene $K/N < 0.001$ y $B = 64$, el número esperado es menor que 0.064 por lote. Además, $\Pr(\text{al menos un crítico}) \leq \mathbb{E}[\text{críticos por lote}]$, así que más del 93.6 % de esos lotes no incluiría una intrusión. Las consecuencias de ignorarla tendrían poca presencia en el aprendizaje.

PER aborda esa dilución al dar mayor probabilidad a las transiciones con error de diferencia temporal (TD) alto [2]:

$$\delta_i = y_i - Q(s_i, a_i; \mathbf{w}), \quad p_i = |\delta_i| + \varepsilon, \quad P(i) = \frac{p_i^\alpha}{\sum_{j \in \mathcal{D}} p_j^\alpha}, \quad \varepsilon > 0, \alpha \in [0, 1]. \quad (3)$$

Aquí y_i es el target temporal de la transición i . Si una intrusión poco frecuente produce un error TD grande, PER permite reutilizarla más que el muestreo uniforme. $\alpha = 0$ recupera el muestreo uniforme; pesos de importancia proporcionales a $(NP(i))^{-\beta}$ pueden compensar el sesgo del nuevo muestreo. PER no crea eventos ausentes ni garantiza priorizar un caso crítico cuyo error TD sea bajo: habría que vigilar además la cobertura de incidentes en el buffer.

1.1(d) Recompensa y falsos positivos

Si se premiara únicamente reducir falsos positivos, la política más sencilla sería *ignorar siempre*: no bloquearía tráfico legítimo, pero dejaría pasar intrusiones. El 8% de falsos positivos del sistema de reglas sirve como contexto, no determina por sí solo los costos de una función de recompensa.

Una propuesta es definir $z_t = 1$ cuando se confirma una intrusión, $h_t \in [0, 1]$ como su gravedad, $e_t \in [0, 1]$ como fracción contenida y $m_t \in [0, 1]$ como fracción del daño que se materializó. Para la acción a_t :

$$r_t = \underbrace{20z_th_te_t}_{\text{contención útil}} - \underbrace{1000z_th_tm_t}_{\text{daño por intrusión}} - \underbrace{(1 - z_t)C_{\text{FP}}(a_t)}_{\text{intervención sobre tráfico legítimo}} - \underbrace{C_{\text{op}}(a_t)}_{\text{costo operativo}}. \quad (4)$$

Acción a	$C_{\text{FP}}(a)$	$C_{\text{op}}(a)$
Ignorar	0	0
Alertar	2	0.2
Bloquear temporalmente	6	1
Aislar segmento	20	3

Estos números son **ejemplos para discutir prioridades, no parámetros validados**. El daño de una intrusión grave recibe el mayor peso; una intervención errónea cuesta más cuanto más interrumpe la operación; y el costo operativo evita acciones innecesarias incluso si hay ataque. La contención efectiva aporta un beneficio menor que la pérdida asociada al daño grave. En una implementación habría que definir cómo observar e_t y m_t , que pueden conocerse con retraso, y ajustar la escala con especialistas y datos de incidentes. La recompensa debe favorecer decisiones según el riesgo del estado, no una acción fija para todos los pasos.

2. Task 1.2: análisis de Double DQN

1.2(a) Sobreestimación y consecuencia operacional

Supongamos que las estimaciones de una red son $\hat{Q}(s', a) = Q(s', a) + \epsilon_a$, con error medio cero para cada acción. Como el máximo es una función convexa,

$$\mathbb{E}[\max_a \hat{Q}(s', a)] \geq \max_a \mathbb{E}[\hat{Q}(s', a)] = \max_a Q(s', a). \quad (5)$$

El máximo tiende a escoger errores positivos junto con acciones aparentemente buenas. En DQN la misma red objetivo selecciona y evalúa la acción del siguiente estado, de modo que ese valor optimista entra al target y puede propagarse mediante *bootstrapping* [2]. Double DQN reduce esta fuente de sesgo al separar selección y evaluación; no elimina todo error de estimación.

Si en estados relevantes se sobrevalora *bloquear*, el agente podría bloquear tráfico legítimo y causar interrupciones y falsos positivos. Si se sobrevalora *ignorar*, podría omitir ataques, retrasar la respuesta

y permitir mayor daño. La sobreestimación del máximo no implica que todas las acciones estén sobreestimadas en cada estado: estas son consecuencias posibles cuando el error favorece una acción concreta.

1.2(b) Targets y cambio exacto

Para una transición (s, a, r, s', d) , donde $d = 1$ si el episodio terminó, sea $\mathbf{w}$ la red en línea y $\mathbf{w}^-$ la red objetivo. Los targets son:

$$y_{\text{DQN}} = r + \gamma(1 - d) \max_{a' \in \mathcal{A}} Q(s', a'; \mathbf{w}^-), \quad (6)$$

$$a_{\text{online}}^* = \arg \max_{a' \in \mathcal{A}} Q(s', a'; \mathbf{w}),$$

$$y_{\text{Double}} = r + \gamma(1 - d) Q(s', a_{\text{online}}^*; \mathbf{w}^-). \quad (7)$$

En (6), $\mathbf{w}^-$ tanto selecciona mediante $\max$ como evalúa. En (7), $\mathbf{w}$ solo selecciona a_{online}^* y $\mathbf{w}^-$ evalúa esa acción. En ambos casos se ajusta $\mathbf{w}$ para aproximar $Q(s, a; \mathbf{w})$ al target, por ejemplo minimizando $(y - Q(s, a; \mathbf{w}))^2$, y se sincroniza periódicamente la red objetivo. El único cambio algorítmico de implementación es el cálculo del target: reemplazar el máximo evaluado por la red objetivo por una selección con la red en línea y una evaluación con la red objetivo. No se agrega una tercera red [2].

1.2(c) Estados ambiguos y claramente maliciosos

En tráfico *borderline*, los valores reales de varias acciones pueden estar próximos. Entonces un error pequeño puede cambiar cuál parece mejor, y el máximo de DQN puede amplificar la estimación optimista de una acción costosa. El riesgo es especialmente delicado cuando la decisión alterna entre ignorar y bloquear: ambos errores tienen consecuencias operativas distintas.

En estados claramente maliciosos, si una acción de contención supera ampliamente a las demás en valor real, errores pequeños tienen menos probabilidad de cambiar la acción elegida. Puede seguir habiendo sobreestimación del valor numérico, pero su efecto sobre la decisión suele ser menor. Esta comparación depende de la separación entre valores, la incertidumbre de las observaciones y los costos; no demuestra que todos los estados maliciosos sean fáciles de clasificar.

Referencias

- [1] CC3104 – Reinforcement Learning. *Laboratorio 7*, guía de la actividad, 2026, Task 1, pp. 1–2.
- [2] CC3104 – Reinforcement Learning. *Deep RL Parte 1: DQN y Variantes*, diapositivas de clase, 2026, pp. 3–5, 8 y 10.